

QE 실습을 slash command로 안전하게

OpenCode 안에서 Quantum
ESPRESSO/XSpectra 작업을 확인, 계획,
제출, 기록하는 교육용 플러그인입니다.

check → plan → submit →
plot

로그인 노드 안전 규칙과 랩 노트 기록을 기본값으로 둡니다.
PDF version

초보자가 위험한 명령을 바로 실행하지 않도록 계산 절차를 좁혀 줍니다.

명령어 허브 >

/qe 계열 slash command로 자주 쓰는 QE 작업을 같은 방식으로 시작합니다.

스케줄러 우선 >

allocation 없이 pw.x, xspectra.x를 로그인 노드에서 직접 실행하지 않게 막습니다.

랩 노트 >

제출, plot, debug 기록을 .oh-my-qe/lab/에 남겨 수업 후에도 추적합니다.

●● 설치

OpenCode를 준비한 뒤 Oh My QE를 설치합니다.

```
$ git clone https://github.com/material-JH/oh-my-qe.git
$ ./oh-my-qe/install.sh
$ export PATH="$HOME/.local/bin:$PATH"
$ cd /scratch/$USER/xspectra-tutorial
$ opencode
```

신규 데스크톱 앱 베타 버전 출시 macOS, Windows, Linux 지원. 지금 다운로드

오픈 소스 **AI** 코딩 에이전트

무료 모델이 포함되어 있으며, 어떤 제공자의 모델이든 연결 가능합니다.
Claude, GPT, Gemini 등을 포함합니다.

curl npm bun brew paru

curl -fsSL https://opencode.ai/install | bash

설치 후 OpenCode의  palette에서 qe 명령을 찾습니다.

자주 쓰는 명령만 top-level에 두고, 나머지는 /qe hub로 묶었습니다.

```
/qe      Oh My QE command hub
/qe-help Show Oh My QE command guide
/qe-plan Build a beginner-safe QE run plan
/qe-plot Run an existing QE plotting script
/qe-debug Inspect recent QE and scheduler output
/qe-check Check QE and queue readiness
/qe-submit Submit an existing Slurm or PBS script safely
/qe-status Show current Slurm or PBS queue status
/qe-lab-view View or search Oh My QE lab notebook
/qe-lab-init Initialize Oh My QE lab notebook
```

```
/qe█
```

```
Build · Big Pickle OpenCode Zen
```

```
tab agents ctrl+p commands
```

● **Tip** Run `/connect` to add an AI provider and start coding

먼저 쓰는 4개

- `/qe-help` — 명령어 안내
- `/qe-check` — 환경 점검
- `/qe-plan` — 실행 계획
- `/qe-submit` — queue 제출

화면에는 간단히 보이고, 세부 기능은 hub subcommand로 들어갑니다.

Oh My QE 슬래시 명령어 안내

최상위 슬래시 명령어 (/qe 타이핑 시 표시, 최대 10개)

명령어	설명
/qe	덜 자주 쓰는 워크플로를 모아둔 허브
/qe-help	이 안내를 출력
/qe-check [DIR]	QE 실행파일, Python, 스케줄러 준비 상태 확인
/qe-plan [DIR]	안전한 실행 계획 수립 (무거운 작업은 실행 안 함)
/qe-submit <SCRIPT> [DIR]	.sbatch / .slurm / .pbs 스크립트 제출
/qe-status	현재 큐 작업 조회
/qe-debug [DIR]	최근 출력 로그 확인 후 오류 원인 설명
/qe-plot [DIR]	기존 플롯 스크립트 실행
/qe-lab-init [DIR]	마크다운 연구 노트 생성
/qe-lab-view [DIR] [QUERY]	연구 노트 보기/검색

서브커맨드	설명
resources [pseudo structure cif all]	슈도퍼텐셜/구조 DB 링크 출력
files [DIR]	프로젝트 파일 목록
env [DIR]	환경 요약 (호스트, 스케줄러, MPI, QE, Python, 디스크)
mode [expert beginner]	응답 모드 전환
expert <REQUEST>	전문가 모드로 요청 실행
explain-input <FILE>	QE 입력 파일 설명
validate-input <FILE DIR>	입력 파일 정적 검사 (BLOCKER/WARN)
defect-plan [DIR]	결함 계산 계획 수립
xspectra-plan [DIR]	XSpectra 계산 계획 수립
lab-update [DIR]	연구 노트 갱신

계산을 실행하기 전에 환경, 입력, queue를 먼저 닫습니다.

`/qe-check`

QE executable, Python, scheduler, project write permission 확인



`/qe-lab-init`

계산 기록을 남길 markdown lab notebook 생성



`/qe-plan`

무거운 작업을 실행하지 않고 필요한 파일과 순서만 정리



`/qe-submit`

검증된 `.sbatch/.pbs` script만 queue로 제출



원본 계산 파일은 그대로 두고, 해석과 진행 상황은 노트에 남깁니다.

랩 노트북은 `/home01/e1956a02/.oh-my-qe/lab/`에 위치해 있습니다.

- `index.md` - 중요 랩 페이지 목록 (카탈로그)
- `notes.md` - 현재까지의 종합/해석 노트
- `log.md` - 추가 전용 시계열 기록 (스케줄러/플롯/QE 활동 자동 기록)

원시 QE 입력(`*.in`) 및 출력 파일은 **불변의 진실 공급원(source of truth)**이고, `.oh-my-qe/lab/`은 그 위에 쌓는 **유지보수된 합성 계층(maintained synthesis layer)**입니다. 계산을 제출/분석/플롯할 때마다 랩 노트를 업데이트하게 됩니다.

생성되는 파일 >

- `index.md` — 중요한 페이지 목록
- `notes.md` — 현재 synthesis
- `log.md` — 제출/plot/debug 기록

첫 예제는 Diamond C K-edge로 전체 흐름만 확인합니다.

1. 확인된 계산 유형 (Diamond C K-edge XANES)

단 계	입 력 파 일	실 행 파 일	설 명
1a	diamond.scf.in	pw.x	SCF (core-hole 없음)
1b	diamond.xspectra.in	xspectra.x	XANES (점 유 상 태 포 함)
1c	diamond.xspectra_replot.in	xspectra.x	XANES 재 플롯 (점 유 상 태 제 거)
1d	diamonhd.scf.in	pw.x	SCF (core-hole 있음, tot_charge=+1.0)
1e	diamonhd.xspectra.in	xspectra.x	XANES (core-hole, 점 유 상 태 제 거)

SrTiO3 O K-edge 예제도 함께 제공되어 있습니다.

2. 제출할 스케줄러 스크립트

- **Slurm:** scheduler/slurm_diamond.sbatch (sbatch scheduler/slurm_diamond.sbatch)
- **PBS:** scheduler/pbs_diamond.pbs (qsub scheduler/pbs_diamond.pbs)

3. 성공 지표 (출력 파일)

- diamond.scf.out - final ! 확인
- diamond.xspectra.out / diamond.xspectra_replot.out / diamonhd.xspectra.out - JOB DONE 확인

학생이 확인할 것

- 입력 파일과 실행 파일의 대응
- Slurm/PBS script 위치
- JOB DONE 확인 파일
- plot script 실행 위치

●● 결과 확인

제출은 기록되고, 완료 뒤에는 기존
plot script로 그림을 만듭니다.

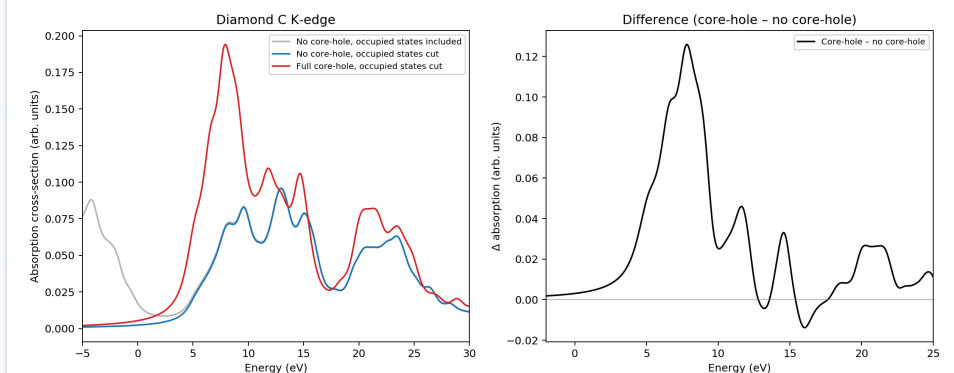
← Edit .oh-my-qe/lab/log.md

```
8 - Pseudopotentials bundled in 'diamond/pseudo/' (C_PBE_TM_2pj.UPF, Ch_PBE_TM_2pj.UPF)
9 - Warning: do not run pw.x/xspectra.x on login node
10 - Lab notebook initialized: index.md, notes.md, log.md created
11
12 + ## 2026-06-25 - Job submitted: Diamond C K-edge XANES
13 +
14 + - Created 'env.sh' for this cluster (intel/19.1.2, impi/19.1.2, qe/7.3.1, python/3.7,
    craype-mic-knl)
15 + - Submitted: 'qsub /scratch/e1956a02/xspectra-tutorial/scheduler/pbs_diamond.pbs'
16 + - Job ID: 22584109.pbs (gachon queue, 1 node, 16 MPI procs, 30 min walltime)
17 + - Runs 'run_all_examples.sh diamond-only' → 5-step pipeline (SCF no-hole → XSpectra →
    replot → SCF core-hole → XSpectra core-hole)
18 + - Status: QUEUED
19 +
```

완료되었습니다. **qstat**으로 진행 상황을 확인할 수 있습니다:

```
qstat -u $(whoami)
```

작업이 완료되면 **python3 plot_spectra.py** (로그인 노드에서 안전)로 스펙트럼을 시각화합니다.



```
$ qstat -u $(whoami)
$ python3 plot_spectra.py
```

Oh My QE는 계산을 대신 “완성” 하는 도구가 아니라, 안전한 실습 루틴을 고정하는 도구입니다.

1

확인

/qe-check로 환경부터
답습니다.

2

계획

무거운 계산은 plan 후
scheduler로 갑니다.

3

차단

로그인 노드 직접 실행을
피합니다.

4

기록

랩 노트가 다음 분석의
시작점입니다.

이 자료는 설치, 명령어, 안전 규칙, 기록 흐름만 다룹니다.